POLITECNICO SETUBAL

Technical Document 3 – Design of Mechanics

Design e Programação de Jogos (DPJ)

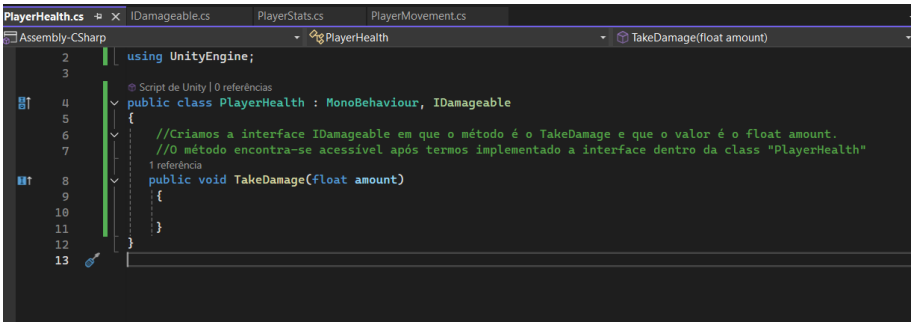
Level 1

We created the PlayerStats script inside the player's scripts folder, converting it from a MonoBehaviour to a ScriptableObject using the [CreateAssetMenu] attribute.

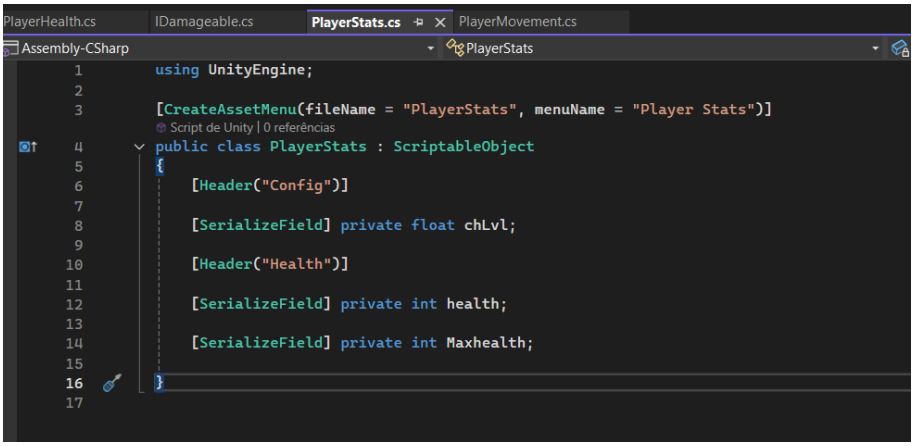


Next, we created an "Extra" folder with the IDamageable interface containing the damage method (TakeDamage).

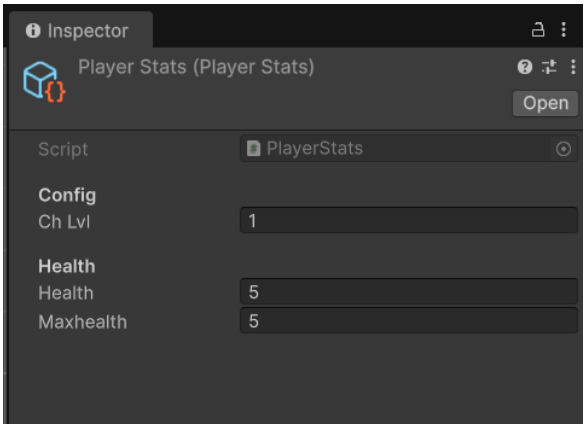




This is the script "PlayerHealth" that has a function called “TakeDamage” with an unset float amout of damage.



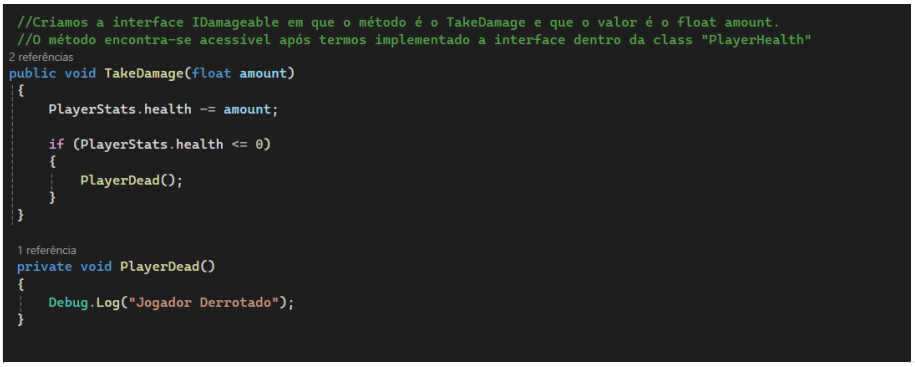
This is the PlayerStats script that we made, that has a few variables such as healt, Maxhealth and chLvl to define the level of the character.



Then, we created the corresponding asset and defined the variables for level, current health, and maximum health.

Level 2

We updated the PlayerHealth script and implemented the IDamageable interface. We added a private reference to PlayerStats visible in the Inspector, coded the health subtraction inside the TakeDamage method, and wrote logical checks to invoke PlayerDead() and log a message to the console (Debug.Log) when health drops to zero or below. We also added a test trigger on the P key inside the Update method.



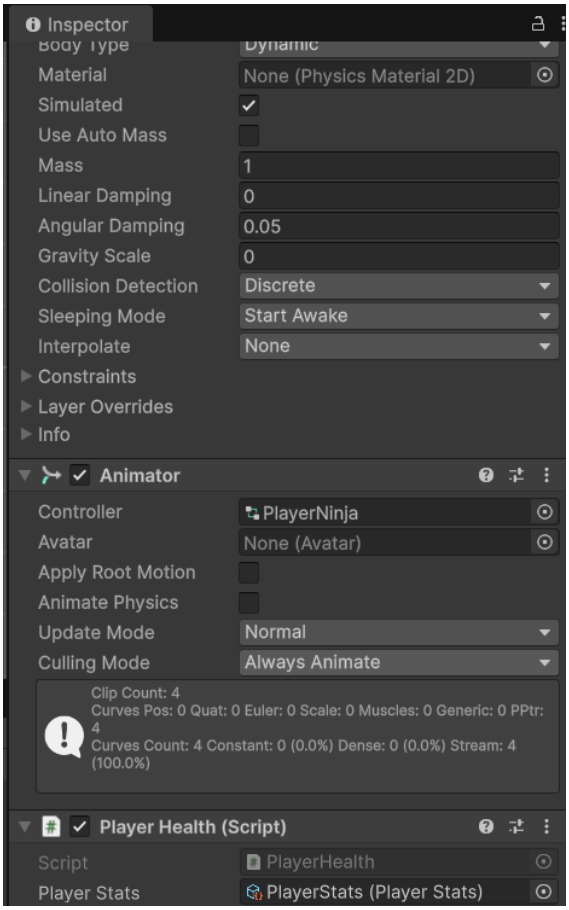
```
//Criamos a interface IDamageable em que o método é o TakeDamage e que o valor é o float amount.
//O método encontra-se acessível após termos implementado a interface dentro da class "PlayerHealth"
2 referências
public void TakeDamage(float amount)
{
    PlayerStats.health -= amount;

    if (PlayerStats.health <= 0)
    {
        PlayerDead();
    }
}

1 referência
private void PlayerDead()
{
    Debug.Log("Jogador Derrotado");
}
```

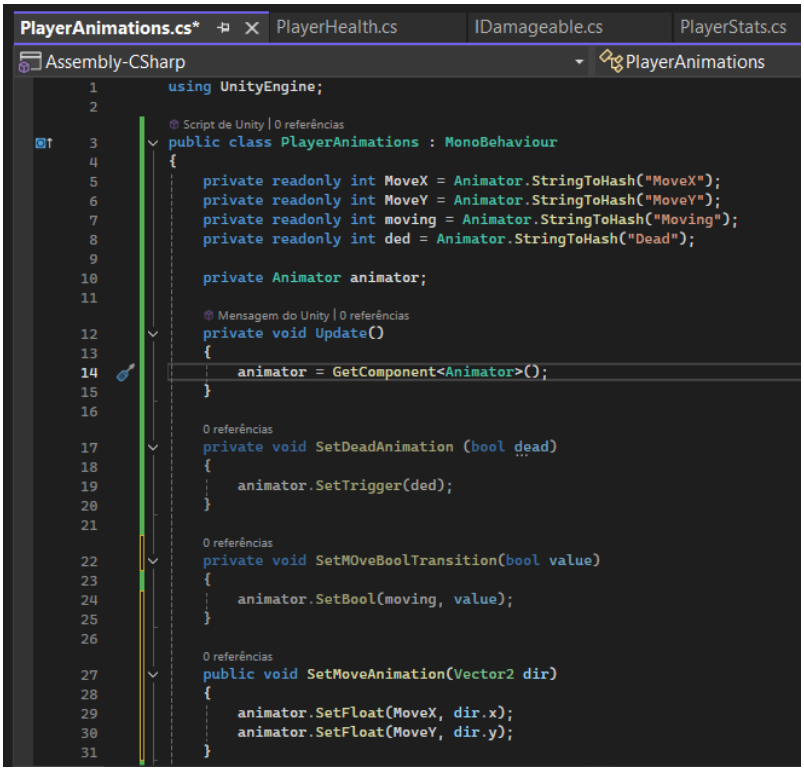
Commented [DM1]: @Martim Santos Pires ADD A NEW IMAGE YOU BUM

Commented [MP2R1]: fawk no



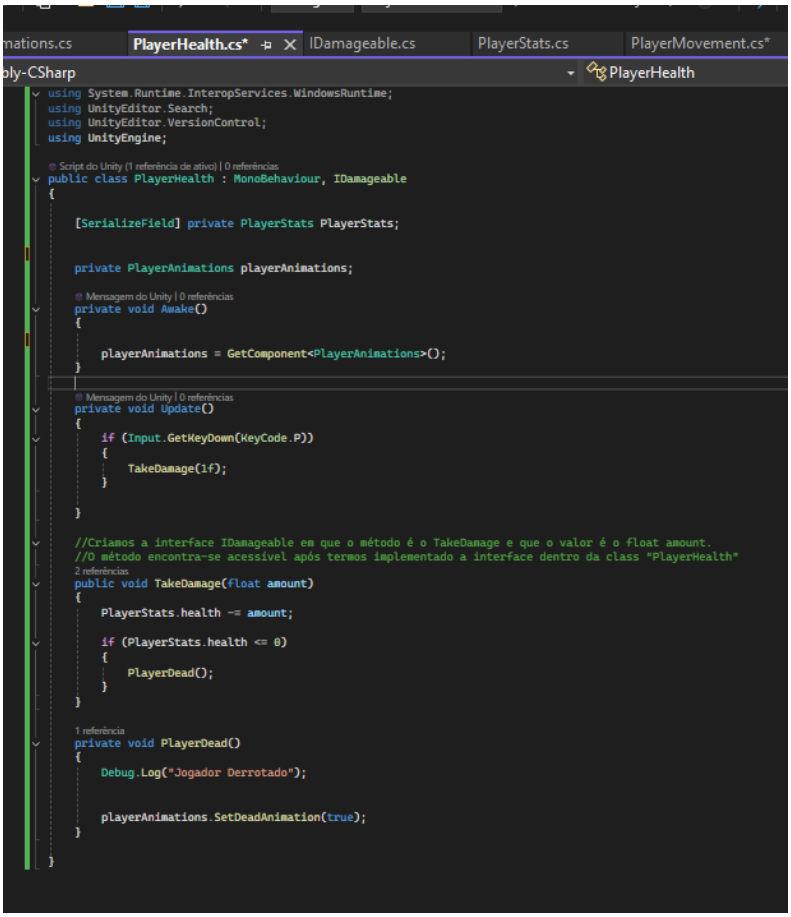
In here is the inspector for the player character which has the PlayerHealth script inside it, with the Player Stats scriptable object inside, which has in it the current health the player has and the max health the player can have, including its level.

Level 3



We configured the death animation (Player_Dead) in Unity's Animator using a Trigger parameter named "Ded" with direct transitions. We created the PlayerAnimations script to manage the Animator using an optimized integer hashes and dedicated methods for movement and death states (SetMoveBoolTransition, SetMoveAnimation, and SetDeadAnimation).

Level 4



We updated the "PlayerMovement" script to reference "PlayerAnimations" and use the transitions we built in the previous step. In the PlayerHealth script, we added the call to trigger the death animation (SetDeadAnimation(true)) whenever the player loses all health.

```
1 referência
private void ReadMovement()
{
    moveDirection = actions.Movement.Move.ReadValue<Vector2>().normalized;

    if (moveDirection == Vector2.zero)
    {

        playerAnimations.SetMoveBoolTransition(false);
        return;
    }


    playerAnimations.SetMoveBoolTransition(true);
    playerAnimations.SetMoveAnimation(moveDirection);
}
```

Level 5

Player.cs | PlayerAnimations.cs | PlayerHealth.cs | IDamageable.cs | PlayerStats.cs | PlayerMovement.cs

Assembly-CSharp | Player

```
1   using UnityEngine;
2
    Script de Unity | 2 referências
3   public class Player : MonoBehaviour
4   {
5
6       [SerializeField] private PlayerStats playerStats;
7
        1 referência
8       public PlayerStats Stats => playerStats;
9   }
```

```
1 referência
private void Move()
{

    if (player.Stats.health <= 0)
    {
        return;
    }

    rb.MovePosition(rb.position + moveDirection * (speed * Time.fixedDeltaTime));
}
```

We created a class named "Player" containing a Lambda property (public PlayerStats Stats => playerStats) to safely expose statistics. In the PlayerMovement script, we added a reference to this component and implemented a validation check inside the method "Move()" to stop character movement if its current health is less than or equal to zero.